

This is unreleased documentation for Serverpod **Next** version.

For up-to-date documentation, see the [latest version](#) (4.0.0).

3.6.7

STREAMING ENDPOINTS (REMOVED IN 4.0)

 Copy as Markdown

Streaming endpoints (removed in 4.0)

WARNING

Streaming endpoints were deprecated in Serverpod 3.0 and removed in 4.0. Use [streaming methods](#) instead. This page documents the old API for projects that have not ported off it yet.

Streaming endpoints were Serverpod's first attempt at streaming data. This approach is more manual, requiring you to manage the WebSocket connection to the server.

Handling streams server-side

The Endpoint class has three methods you override to work with streams.

- `streamOpened` is called when a user connects to a stream on the Endpoint.
- `streamClosed` is called when a user disconnects from a stream on the Endpoint.
- `handleStreamMessage` is called when a serialized message is received from a client.

To send a message to a client, call the `sendStreamMessage` method. You will need to include the session associated with the user.

The user object

Associate state with a streaming session by setting a user object when the stream is opened.

```
Future<void> streamOpened(StreamingSession session) async {  
  setUserObject(session, MyUserObject());  
}
```

You can access the user object at any time by calling the `getUserObject` method. The user object is automatically discarded when a session ends.



Ask AI

Handling streams in your app

Before you can access streams in your client, you need to connect to the server's web socket by calling `openStreamingConnection` on your client.

```
await client.openStreamingConnection();
```

You can monitor the state of the connection by adding a listener to the client. Once connected to your server's web socket, you can pass and receive serialized objects.

Listen to its web socket stream to receive updates from an endpoint on the server.

```
await for (var message in client.myEndpoint.stream) {  
  _handleMessage(message);  
}
```

You send messages to the server's endpoint by calling `sendStreamMessage`.

```
client.myEndpoint.sendStreamMessage(MyMessage(text: 'Hello'));
```

INFO

Authentication is handled automatically. If you have signed in, your web socket connection will be authenticated.